

**LAPORAN PRAKTIKUM PEMROGRAMAN BERORIENTASI OBJEK (PBO)**

**PRAKTIKUM 11**



**Nim : 2411102441125**

**Nama : Tiara Nazwa Andira Ali**

**FAKULTAS SAINS DAN TEKNOLOGI**

**PROGRAM STUDI S1 TEKNIK INFORMATIKA**

**UNIVERSITAS MUHAMMADIYAH KALIMANTAN TIMUR**

**EMAIL : [2411102441125@umkt.ac.id](mailto:2411102441125@umkt.ac.id)**

## Langkah 1: Gabungan Dari Langkah 1,2,3

```

Langkah 123.py X
Langkah 123.py > _
1 from abc import ABC, abstractmethod
2 from dataclasses import dataclass
3
4 @dataclass
5 class Order:
6     customer_name: str
7     total_price: float
8     status: str = "open"
9
10 class IPaymentProcessor(ABC):
11     """Kontrak: Semua prosesor pembayaran harus punya method 'process'."""
12     @abstractmethod
13     def process(self, order: Order) -> bool:
14         pass
15
16 class INotificationService(ABC):
17     """Kontrak: Semua layanan notifikasi harus punya method 'send'."""
18     @abstractmethod
19     def send(self, order: Order):
20         pass
21
22 class CreditCardProcessor(IPaymentProcessor): # Tanggung jawab tunggal: Memproses CC
23     def process(self, order: Order) -> bool:
24         print("Payment: Memproses Kartu Kredit.")
25         return True
26
27 class EmailNotifier(INotificationService): # Tanggung jawab tunggal: Mengirim Email
28     def send(self, order: Order):
29         print(f"Notif: Mengirim email konfirmasi ke {order.customer_name}.")
30
31 class CheckoutService:
32     # Tanggung jawab tunggal: Mengkoordinasi alur Checkout (SRP)
33
34     def __init__(self, payment_processor: IPaymentProcessor, notifier: INotificationService):
35         # Dependency Injection (DI): Bergantung pada Abstraksi
36         self.payment_processor = payment_processor
37         self.notifier = notifier
38
39     def run_checkout(self, order: Order):
40         payment_success = self.payment_processor.process(order) # Delegasi ke Abstraksi
41
42         if payment_success:
43             order.status = "paid"
44             self.notifier.send(order) # Delegasi ke Abstraksi
45             print("Checkout Sukses.")
46             return True
47         return False
48
49 if __name__ == "__main__":
50     # Setup Order dan Notifier standar
51     andi_order = Order("Andi", 500000)
52     email_service = EmailNotifier()
53
54     # Skenario 1: Credit Card
55     cc_processor = CreditCardProcessor()
56     checkout_cc = CheckoutService(payment_processor=cc_processor, notifier=email_service)
57     print("--- Skenario 1: Credit Card ---")
58     checkout_cc.run_checkout(andi_order)
59
60     class QrisProcessor(IPaymentProcessor): # Kelas baru untuk Ekstensi
61         def process(self, order: Order) -> bool:
62             print("Payment: Memproses QRIS.")
63             return True
64
65     budi_order = Order("Budi", 100000)
66     qris_processor = QrisProcessor()
67
68     checkout_qris = CheckoutService(payment_processor=qris_processor, notifier=email_service)
69     print("\n--- Skenario 2: Pembuktian OCP (QRIS) ---")
70     checkout_qris.run_checkout(budi_order)

```

## Output:

```

[Running] python -u "c:\Users\Asus\Downloads\PBO 11,12,13\Langkah 1,2,3.py"
--- Skenario 1: Credit Card ---
Payment: Memproses Kartu Kredit.
Notif: Mengirim email konfirmasi ke Andi.
Checkout Sukses.

--- Skenario 2: Pembuktian OCP (QRIS) ---
Payment: Memproses QRIS.
Notif: Mengirim email konfirmasi ke Budi.
Checkout Sukses.

[Done] exited with code=0 in 0.17 seconds

```

## Studi Kasus: Eksekusi Dan Pembuktian OCP

```

(Latihan Mandiri) Eksekusi Dan Pembuktian OCP.py X
(Latihan Mandiri) Eksekusi Dan Pembuktian OCP.py > ...
1  from abc import ABC, abstractmethod
2  from dataclasses import dataclass
3
4  # Asumsi kelas Order sudah didefinisikan
5  @dataclass
6  class Order:
7      customer_name: str
8      total_price: float
9      status: str = "open"
10
11  # --- ABSTRAKSI (Kontrak untuk OCP/DIP) ---
12  class IPaymentProcessor(ABC):
13      """Kontrak: Semua prosesor pembayaran harus punya method 'process'."""
14      @abstractmethod
15      def process(self, order: Order) -> bool:
16          pass
17
18  class INotificationService(ABC):
19      """Kontrak: Semua layanan notifikasi harus punya method 'send'."""
20      @abstractmethod
21      def send(self, order: Order):
22          pass
23
24  # --- IMPLEMENTASI KONKRIT (Plug-in) ---
25  class CreditCardProcessor(IPaymentProcessor):
26      def process(self, order: Order) -> bool:
27          print("Payment: Memproses Kartu Kredit.")
28          return True
29
30  class EmailNotifier(INotificationService):
31      def send(self, order: Order):
32          print(f"Notif: Mengirim email konfirmasi ke {order.customer_name}.")
33
34  # --- KELAS KOORDINATOR (SRP & DIP) ---
35  class CheckoutService:
36      # Tanggung jawab tunggal: Mengkoordinasi Checkout (SRP)
37
38      def __init__(self, payment_processor: IPaymentProcessor, notifier: INotificationService):
39          # Dependency Injection (DIP): Bergantung pada Abstraksi, bukan Konkrit
40          self.payment_processor = payment_processor
41          self.notifier = notifier
42
43      def run_checkout(self, order: Order):
44          payment_success = self.payment_processor.process(order) # Delegasi 1
45
46          if payment_success:
47              order.status = "paid"
48              self.notifier.send(order) # Delegasi 2
49              print("Checkout Sukses.")
50              return True
51          return False
52
53
54  # --- PROGRAM UTAMA ---
55  if __name__ == "__main__":
56      # Setup Dependencies
57      andi_order = Order("Andi", 500000)
58      email_service = EmailNotifier()
59
60      # 1. Inject implementasi Credit Card
61      cc_processor = CreditCardProcessor()
62      checkout_cc = CheckoutService(payment_processor=cc_processor, notifier=email_service)
63      print("--- Skenario 1: Credit Card ---")
64      checkout_cc.run_checkout(andi_order)
65
66      # 2. Pembuktian OCP: Menambah Metode Pembayaran QRIS (Tanpa Mengubah CheckoutService)
67      class QrisProcessor(IPaymentProcessor):
68          def process(self, order: Order) -> bool:
69              print("Payment: Memproses QRIS.")
70              return True
71
72      budi_order = Order("Budi", 100000)
73      qris_processor = QrisProcessor()
74
75      # Inject implementasi QRIS yang baru dibuat
76      checkout_qris = CheckoutService(payment_processor=qris_processor, notifier=email_service)
77      print("\n--- Skenario 2: Pembuktian OCP (QRIS) ---")
78      checkout_qris.run_checkout(budi_order)

```

**Output:**

```
[Running] python -u "c:\Users\Asus\Downloads\PBO 11,12,13\{Latihan Mandiri} Eksekusi Dan Pembuktian OCP.py"
--- Skenario 1: Credit Card ---
Payment: Memproses Kartu Kredit.
Notif: Mengirim email konfirmasi ke Andi.
Checkout Sukses.

--- Skenario 2: Pembuktian OCP (QRIS) ---
Payment: Memproses QRIS.
Notif: Mengirim email konfirmasi ke Budi.
Checkout Sukses.

[Done] exited with code=0 in 0.175 seconds
```

**Refleksi:** DI membalikkan alur ketergantungan. Kode menjadi terbuka untuk pengembangan (karena Anda bisa membuat kelas baru, seperti QrisProcessor, kapan saja) tetapi tertutup untuk modifikasi (karena kelas koordinator lama tidak perlu disentuh). Kelas koordinator hanya perlu "disuntikkan" dengan implementasi baru saat runtime.